Investigating a Potential TOCTOU in the iOS 26.0.1 MTE Tagging Path

Kaya Ercihan

01.03.2026

Abstract

Memory Tagging Extension (MTE) is increasingly relied upon as a hardening primitive in modern kernels, where tag generation and publication must remain correct under concurrency. This paper presents a static reverse-engineering assessment of the MTE tag-generation path in the iOS 26.0.1 kernel (Darwin 24.x) targeting A19-class SoCs (iPhone 17 series). We analyze the helper routine `irg_mte_function` and its direct caller contexts with the goal of identifying missing synchronization that could permit a time-of-check to time-of-use (TOCTOU) race (CWE-362) during tag creation and pointer publication. Using IDA Pro 9.1 and custom IDA Python tooling, we extract full disassembly, enumerate direct call sites, and scan for explicit ordering barriers (`DMB`/`DSB`/`ISB`) and lock/atomic primitives. Our results show that `irg_mte_function` and a downstream tagging worker (`sub_FFFFFE000843B738`) contain no explicit ordering barriers. Path-sensitive analysis of the 12 direct call sites, resolved to 11 enclosing caller functions, identified a single barrier-bearing caller context, `sub_FFFFFE0008331030`, containing an explicit `DMB ISH`. The remaining analyzed helper-reaching paths contained no `DMB`, `DSB`, or `ISB` on the traced helper-to-update/publication segment. These paths contained atomic read–modify–write operations, including acquire- and release-qualified variants, but no complete acquire/release publication protocol or lock-based critical section spanning both the helper invocation and the corresponding shared-state publication was identified. We further characterize trigger preconditions in the call graph via a per-context flag gate (byte at offset `+0x3C`) and a global enable/count parameter (`dword_FFFFFE0007C0E250`) with both read and write references. Although no dynamic validation was possible due to lack of SRD/jailbroken hardware, the static analysis indicates that ordering across the tag-store–publication boundary is not consistently established at the direct-caller layer. A security-relevant TOCTOU window remains conditional on concurrent publication and consumption and on the absence of higher-level synchronization outside the traced paths.

1 Introduction

A potential Time-of-Check to Time-of-Use (TOCTOU) race condition has been identified in the Memory Tagging Extension (MTE) tag-generation path of the iOS 26.0.1 kernel running on A19-class SoCs, such as those found in the iPhone 17 series. Static analysis indicates that the function `irg_mte_function` (analyst-assigned IDA Pro name) and most of its caller functions execute without explicit memory barriers ("DMB", "DSB", "ISB") or full critical-section protection, relying solely on atomic read-modify-write operations ("CASA", "CASL", "LDADD", etc.). No complete acquire/release publication protocol

or lock-based critical section spanning the helper and the corresponding shared-state publication was identified. This may allow concurrent CPU cores to observe partially initialized or stale tagged pointers, creating a theoretical TOCTOU window during tag creation and publication.

1.1 Technical overview

- Candidate Issue Type: Race Condition, Time-of-Check to Time-of-Use (TOCTOU)
- CWE-362: Concurrent Execution using Shared Resource with Improper Synchronization [2]
- Suspected Root Cause: Missing full memory-ordering barriers or locks in the MTE tag-generation and pointer publication sequence.
- Impact: Possible inconsistency or corruption of tagged pointers observable by other cores or DMA agents, leading to theoretical exposure of kernel memory or invalid tag faults.
- Attack Vector: Local (requires kernel code execution or ability to trigger concurrent MTE-tagging operations on multiple cores).
- Privileges Required: Kernel-level (no user-space exploit path shown).
- Exploitation Likelihood: Low (requires concurrency on MTE hardware), but indicates incomplete synchronization in security-critical code.

1.2 Analyzed Scope and Potential Applicability

Table 1: Analyzed Scope and Potential Applicability

Product	**Version**	**Architecture**	**Status**
iOS Kernel	26.0.1 (Darwin 24.x)	A19 (ARMv9.2-A) / iPhone 17 Series	Analyzed
iPadOS Kernel	26.0.1	A19 or newer	Not analyzed
macOS (Sonoma/Sequoia)	Not analyzed	M3 / M4 (ARMv9)	Not analyzed

1.3 Strong indication

- **Static indication:** IDA Pro 9.1 disassembly of `irg_mte_function` shows no `DMB`, `DSB`, or `ISB` barriers.
- **Caller Verification:** The 12 direct call sites resolve to 11 enclosing caller functions. Only one (`sub_FFFFFE0008331030`) contains any explicit barrier (`DMB ISH`); all others rely solely on atomic instructions.
- **Dynamic Validation:** Not performed; requires MTE-capable hardware (A19 / iPhone 17).
- **Reproducibility:** Can be statically verified on any extracted iOS 26.0.1 kernel image using the provided IDA Python scripts.

1.4 Detection Methodology

- **Tools Used:** IDA Pro 9.1, custom Python scripts.
- **Analysis Type:** Static code inspection, call-graph traversal, synchronization primitive detection.
- **Result:** 11 of the 12 direct call sites lack an explicit `DMB`, `DSB`, or `ISB` on the traced helper-to-update/publication path segment.

1.5 Potential Mitigations if Missing Ordering Is Confirmed

- Insert explicit **memory barriers** (`DMB ISH` / `DSB SY`) before publishing tagged pointers.
- Enforce **critical-section locking** (e.g., spinlocks) around tag generation and pointer publication.
- Review the MTE helper (`irg_mte_function`) to include a global ordering fence or enforce tag initialization under lock.
- Validate concurrency safety on A19 hardware using Apple's internal kernel stress tools or SRD devices.

1.6 Research Context

This report originates from a static kernel analysis of iOS 26.0.1 (Darwin 24.x) targeting the A19 (ARMv9.2-A) architecture.
No exploitation or live testing was performed.
Dynamic verification would require MTE-capable hardware such as the iPhone 17 or access to an Apple Security Research Device (SRD).

2 Methodology

To obtain strong indication that a time-of-check to time-of-use (TOCTOU) condition is feasible, it is necessary to confirm that the tag creation on the relevant memory region is not constrained by intervening synchronization or ordering mechanisms. This assessment requires first recovering and examining the complete instruction sequence of the corresponding function (or basic) block to identify any serialization, synchronization, or other ordering-enforcing operations that would prevent a check/use window from arising.

2.1 Study Design

We conducted a static reverse-engineering study of the iOS 26.0.1 (Darwin 24.x) kernel to assess whether the Memory Tagging Extension (MTE) tag-generation and pointer publication path enforces sufficient inter-core memory ordering. The analysis focuses on identifying missing explicit ordering primitives (e.g. `DMB`/`DSB`/`ISB`) or critical-section protection that could permit a time-of-check to time-of-use (TOCTOU) window under concurrent execution.

2.2 Scope and Targets

The primary target routine is `irg_mte_function` (IDA Pro symbol name), identified as an MTE tag-generation helper on iOS 26.0.1. The secondary target set comprises all direct call sites to this routine, defined as code references that invoke `irg_mte_function` via branch-and-link instructions (`BL`) and their enclosing caller functions.

2.3 Tooling

Static analysis was performed using IDA Pro 9.1 with custom IDA Python scripts: (i) `dump_current_func_instructions.py` for extracting full function disassembly, (ii) `ida_irg_toctou_check.py` for enumerating direct callers and scanning for synchronization primitives, and (iii) `verify_barrier_paths.py` for automated per-caller classification.

2.4 Analysis Procedure

The analysis consisted of the following steps:

1. **Disassembly extraction:** Extract the complete instruction stream of `irg_mte_function` from the kernel image and record all instructions for subsequent pattern matching.

2. **Caller enumeration:** Enumerate all direct call references to `irg_mte_function` and resolve each reference to its enclosing caller function.

3. **Path reconstruction and synchronization scanning:** For each direct call site, reconstruct the control-flow paths reaching `irg_mte_function` and the subsequent caller-side update or publication sequence. Scan the reachable basic blocks on these path segments for explicit barrier instructions (`DMB`, `DSB`, `ISB`), paired lock operations, calls to lock primitives, and atomic read–modify–write instructions (e.g. `CAS*`, `LDADD*`, `LDXR/STXR`, and `SWP*`). MTE tag-generation and tag-store instructions (e.g. `IRG`, `STG`, `ST2G`, and `STGM`) were recorded for context but were not treated as general-purpose ordering fences.

4. **Path-level ordering assessment:** Determine whether the analyzed tag-store–publication segment contains (i) an explicit architectural barrier, (ii) a lock-based critical section that begins before the helper and extends through the corresponding shared-state update or publication operation, or (iii) a complete acquire/release publication protocol linking the helper's tag stores to that operation. Acquire- and release-qualified atomic instructions were recorded separately, but their presence alone was not treated as proof of a complete end-to-end publication protocol.

5. **Result aggregation:** Classify each analyzed caller path as barrier-bearing, atomic-RMW without an explicit fence on the traced segment, or lacking an identified ordering primitive. Summarize whether ordering across the tag-store–publication boundary is applied consistently across the complete direct-caller set.

2.5 Limitations

This study is based on static analysis only. No dynamic validation (e.g. stress testing on MTE-capable A19 hardware or Apple Security Research Devices) was performed, and therefore the results provide strong indication of potential synchronization weaknesses rather than evidence.

3 Preconditions and Trigger Conditions

This section summarizes the static preconditions required to reach the MTE-tagging call paths and the conditions under which the bulk tagging logic executes. The findings are derived from IDA-based call-site enumeration, pattern scans for the flag-gating logic, and cross-references to a global control variable used across multiple callers.

3.1 Direct trigger: reaching a direct caller of `irg_mte_function`

A necessary precondition is that execution reaches one of the direct call sites to the helper routine `irg_mte_function`. Automated enumeration identified 12 call sites overall, confirming that the helper is used across multiple kernel paths.

```
0xFFFFFE000832A740: BL              irg_mte_function
```

Listing 1: Representative direct call site to the tagging helper `irg_mte_function`

3.2 Gating condition: per-context enable flag at offset `+0x3C` (bit test)

For the bulk/tagging loop paths, callers load a byte from a context structure at offset `0x3C` and apply a bit-test gate (observed as a `TBZ` on bit 6 in the relevant call paths). This flag acts as a per-context enable/disable switch for the subsequent range-processing behavior.

```
; caller: sub_FFFFFE0008329FDC
0xFFFFFE000832A748: LDRB            W8, [X19,#0x3C]   ; load per-context flag byte
; followed by a bit-test gate in the same control-flow region (bit 6)
```

Listing 2: Observed flag read at structure offset `+0x3C` in a direct caller (gating flag source)

3.3 Global gating/count parameter: `dword_FFFFFE0007C0E250`

In addition to the per-context flag, multiple callers reference a global variable at `0xFFFFFE0007C0E250`. This value is used as a global enable/count parameter: if the value is zero, the bulk processing loop is skipped; if nonzero, it determines whether (and typically how often) the follow-up tagging worker is executed. The variable has extensive read-side usage (169 cross-references were identified).

```
; example within caller: sub_FFFFFE0008329FDC
0xFFFFFE000832A720: ADRP            X20, #dword_FFFFFE0007C0E250@PAGE
0xFFFFFE000832A750: LDR             W8,  [X20,#dword_FFFFFE0007C0E250@PAGEOFF]
```

Listing 3: Caller-side access to the global control/count variable `dword_FFFFFE0007C0E250`

Importantly, at least one kernel routine writes this global value, indicating that it is not merely a constant but can be configured at runtime or initialization time.

```
; writer: sub_FFFFFE0008AD046C
0xFFFFFE0008AD049C: ADRP            X20, #dword_FFFFFE0007C0E250@PAGE
0xFFFFFE0008AD04A0: STR             W8,  [X20,#dword_FFFFFE0007C0E250@PAGEOFF]
```

Listing 4: Write to `dword_FFFFFE0007C0E250` observed in `sub_FFFFFE0008AD046C`

3.4 Bulk tagging worker executes without explicit barriers

If both gating conditions are satisfied (per-context flag enabled and global parameter nonzero), the call paths invoke a follow-up worker routine (`sub_FFFFFE000843B738`) that performs MTE tag-store operations. A pattern scan of `sub_FFFFFE000843B738` shows multiple MTE tag-store instructions and no explicit ordering barriers (`DMB/DSB/ISB`) within the routine, consistent with barrier-free tag-store sequencing at the helper level.

```
0xFFFFFE000843B77C: STG             X9, [X9],#0x10
0xFFFFFE000843B7A4: STGM            X10, [X9]
0xFFFFFE000843B7DC: ST2G            X9, [X9],#0x20
0xFFFFFE000843B7E0: ST2G            X9, [X9],#0x20
0xFFFFFE000843B814: ST2G            X8, [X8],#0x20
0xFFFFFE000843B818: ST2G            X8, [X8],#0x20
0xFFFFFE000843B828: STG             X8, [X8],#0x10
```

Listing 5: MTE tag-store operations inside `sub_FFFFFE000843B738` (no barrier instructions found in this routine)

3.5 Summary of preconditions

The static analysis supports the following trigger/precondition set:

- **Control-flow reachability:** execution must reach a direct caller that performs `BL irg_mte_function` (Listing 1).
- **Per-context enable:** the caller must observe the per-context flag (byte at offset `+0x3C`) in an enabled state; this value is used in a bit-test gate to decide whether bulk processing proceeds (Listing 2).
- **Global enable/count:** the global parameter `dword_FFFFFE0007C0E250` must be nonzero; it is read by multiple callers (Listing 3) and is written by at least one routine (Listing 4).
- **Lack of consistent caller-path ordering:** the helper routines `irg_mte_function` and `sub_FFFFFE000843B738` contain no explicit ordering barriers (`DMB/DSB/ISB`) in their bodies (cf. Listing 5 and Listing 6). Across the analyzed direct-caller paths, only the resolved caller context `sub_FFFFFE0008331030` was observed to contain an explicit `DMB ISH`. The remaining traced helper-to-update/publication path segments contained no explicit architectural fence. Although these paths contain atomic read–modify–write instructions, including acquire- and release-qualified variants, no complete acquire/release publication protocol or lock-based critical section spanning both the helper invocation and the corresponding shared-state publication was identified. Consequently, the direct-caller paths do not consistently establish a barrier-, lock-, or identified acquire/release-based happens-before relation across the tag-store–publication boundary. Any additional ordering would therefore have to be supplied by higher-level synchronization outside the traced paths.

3.6 Threat model assumptions

The conditions above describe *reachability* and *gating* of the relevant MTE-tagging paths. For the suspected TOCTOU window to be security-relevant, additional assumptions must hold: (i) two or more execution contexts (e.g. different CPU cores, interrupt contexts, or preempted threads) can execute the affected caller paths concurrently, (ii) the tagged pointer

value (or associated metadata derived from the tagging sequence) is published to shared memory and may be consumed by a concurrent context before initialization completes, and (iii) no higher-level serialization (e.g. a lock discipline or acquire/release protocol) reliably enforces a happens-before relationship across the publication/consumption boundary. The present work provides static strong indication of missing explicit barrier-based ordering at the helper boundary, but does not claim exploitability without dynamic validation under realistic scheduling and workload conditions.

4 Results and Discussion

This section reports the static findings for the MTE helper routine `irg_mte_function` and its direct caller contexts. We enumerated all direct caller paths to this helper and analyzed each call site and its enclosing function for explicit synchronization, in particular the presence of ordering barriers (`DMB`, `DSB`, `ISB`) or lock-based critical sections. The goal is to determine whether tagged pointer publication is consistently preceded by an architectural ordering primitive, or whether call paths rely primarily on atomic operations without an explicit global ordering guarantee.

4.1 Definitions and interpretation of synchronization indicators

In this work, we distinguish between explicit ordering barriers, atomic read–modify–write (RMW) operations, lock-based critical sections, and MTE/tagging instructions. We treat `DMB`, `DSB`, and `ISB` as explicit architectural ordering primitives. Atomic RMW instructions were recorded according to whether their instruction forms were acquire-qualified, release-qualified, or unqualified. Such instructions can contribute to memory ordering when they participate in a complete synchronization protocol between a producer and a consumer. Their presence alone, however, does not establish that the helper's tag stores are ordered before a distinct pointer or metadata publication operation.

The caller-path assessment therefore searched for an end-to-end relation in which an explicit barrier, a paired lock-based critical section, or a complete acquire/release publication protocol spans the tag-store–publication boundary. A race condition remains possible only if publication and consumption can overlap and if no higher-level kernel invariant supplies the required synchronization outside the traced path. Finally, MTE instructions such as `IRG`, `STG`, `ST2G`, `STGM`, and `LDG` were recorded for context but were not treated as general-purpose ordering fences.

4.2 Helper-level findings

Listing 6 shows a representative excerpt of `irg_mte_function` featuring tag generation (`IRG`) and tag-store sequences (`STG`/`ST2G`/`STGM`). No explicit ordering barriers (`DMB`/`DSB`/`ISB`) were observed within the helper body in the inspected disassembly, implying that any required publication ordering must be provided by the caller context (e.g. via a lock/unlock protocol or an explicit barrier) rather than by the helper itself.

```
0xFFFFFE000843B638: BTI             c
0xFFFFFE000843B63C: IRG             X0, X0, X2
0xFFFFFE000843B640: ADD             X8, X0, X1
0xFFFFFE000843B644: EOR             X16, X0, X8
0xFFFFFE000843B648: TST             X16, #0xFFC0000000000000
0xFFFFFE000843B64C: B.EQ            loc_FFFFFE000843B654
0xFFFFFE000843B650: MOVK            X8, #0xC8A2,LSL#48
0xFFFFFE000843B654: AND             X9, X0, #0xFFFFFFFFFFFFFFF0
```

```
0xFFFFFE000843B658: ADD             X8, X8, #0xF
0xFFFFFE000843B65C: AND             X8, X8, #0xFFFFFFFFFFFFFFF0
0xFFFFFE000843B660: SUB             X10, X8, X9
0xFFFFFE000843B664: CMP             X10, #0x3F ; '?'
0xFFFFFE000843B668: B.GT            loc_FFFFFE000843B67C
0xFFFFFE000843B66C: CMP             X10, #0x10
0xFFFFFE000843B670: B.HI            loc_FFFFFE000843B718
0xFFFFFE000843B674: STG             X9, [X9],#0x10
0xFFFFFE000843B678: RET
0xFFFFFE000843B67C: AND             X10, X0, #0xF0
0xFFFFFE000843B680: CBNZ            X10, loc_FFFFFE000843B6D4
0xFFFFFE000843B684: ADD             X10, X9, #0x100
0xFFFFFE000843B688: CMP             X10, X8
0xFFFFFE000843B68C: B.HI            loc_FFFFFE000843B6D4
0xFFFFFE000843B690: UBFX            X10, X0, #0x38, #4 ; '8'
0xFFFFFE000843B694: MOV             X11, #0x1111111111111111
0xFFFFFE000843B698: MUL             X10, X10, X11
0xFFFFFE000843B69C: STGM            X10, [X9]
0xFFFFFE000843B6A0: ADD             X11, X9, #0x200
0xFFFFFE000843B6A4: ADD             X9, X9, #0x100
0xFFFFFE000843B6A8: CMP             X11, X8
0xFFFFFE000843B6AC: B.LS            loc_FFFFFE000843B69C
0xFFFFFE000843B6B0: CMP             X9, X8
0xFFFFFE000843B6B4: B.EQ            locret_FFFFFE000843B714
0xFFFFFE000843B6B8: SUB             X10, X8, X9
0xFFFFFE000843B6BC: CMP             X10, #0x3F ; '?'
0xFFFFFE000843B6C0: B.GT            loc_FFFFFE000843B6D4
0xFFFFFE000843B6C4: CMP             X10, #0x10
0xFFFFFE000843B6C8: B.HI            loc_FFFFFE000843B728
0xFFFFFE000843B6CC: STG             X9, [X9],#0x10
0xFFFFFE000843B6D0: RET
0xFFFFFE000843B6D4: ST2G            X9, [X9],#0x20
0xFFFFFE000843B6D8: ST2G            X9, [X9],#0x20
0xFFFFFE000843B6DC: CMP             X9, X8
0xFFFFFE000843B6E0: B.EQ            locret_FFFFFE000843B714
0xFFFFFE000843B6E4: AND             X9, X9, #0xFFFFFFFFFFFFFFC0
0xFFFFFE000843B6E8: ADD             X10, X9, #0x40 ; '@'
0xFFFFFE000843B6EC: CMP             X10, X8
0xFFFFFE000843B6F0: B.CS            loc_FFFFFE000843B708
0xFFFFFE000843B6F4: DC              GVA, X9
0xFFFFFE000843B6F8: ADD             X10, X9, #0x80
0xFFFFFE000843B6FC: ADD             X9, X9, #0x40 ; '@'
0xFFFFFE000843B700: CMP             X10, X8
0xFFFFFE000843B704: B.CC            loc_FFFFFE000843B6F4
0xFFFFFE000843B708: SUB             X8, X8, #0x40 ; '@'
0xFFFFFE000843B70C: ST2G            X8, [X8],#0x20
0xFFFFFE000843B710: ST2G            X8, [X8],#0x20
0xFFFFFE000843B714: RET
0xFFFFFE000843B718: ST2G            X9, [X9],#0x20
0xFFFFFE000843B71C: SUB             X8, X8, #0x10
0xFFFFFE000843B720: STG             X8, [X8],#0x10
0xFFFFFE000843B724: RET
0xFFFFFE000843B728: ST2G            X9, [X9],#0x20
0xFFFFFE000843B72C: SUB             X8, X8, #0x10
0xFFFFFE000843B730: STG             X8, [X8],#0x10
0xFFFFFE000843B734: RET
```

Listing 6: Disassembly excerpt of `irg_mte_function`

This function block's control flow graph can be seen in the IDA Pro illustration figure 2.

4.3 Caller-path assessment

To evaluate whether ordering is provided by the call sites, we enumerated all direct calls to `irg_mte_function` and reconstructed the helper-reaching control-flow paths together with the subsequent caller-side update or publication sequences. For each traced path, we examined whether the tag-store–publication interval was spanned by (i) an explicit architectural barrier (`DMB`, `DSB`, or `ISB`), (ii) a paired lock-based critical section, or (iii) a complete acquire/release publication protocol. The direct call pattern is shown in Listing 7. The key question is whether each caller path enforces a happens-before relationship between tag initialization, any derived tagged-pointer value, and the corresponding shared-state publication or consumption by another execution context.

```
1 BL irg_mte_function ; irg_mte_function is a tag
```

Listing 7: Direct call site to `irg_mte_function`

As shown in Figure 1, the IDA Pro cross-reference view (*xrefs*) lists all 12 direct call sites to the helper routine `irg_mte_function`. Each entry corresponds to a direct call site. The 12 sites resolve to 11 enclosing caller functions. These caller functions reach the helper via a direct `BL` instruction, providing the complete set of direct callers used for the subsequent barrier/locking classification.

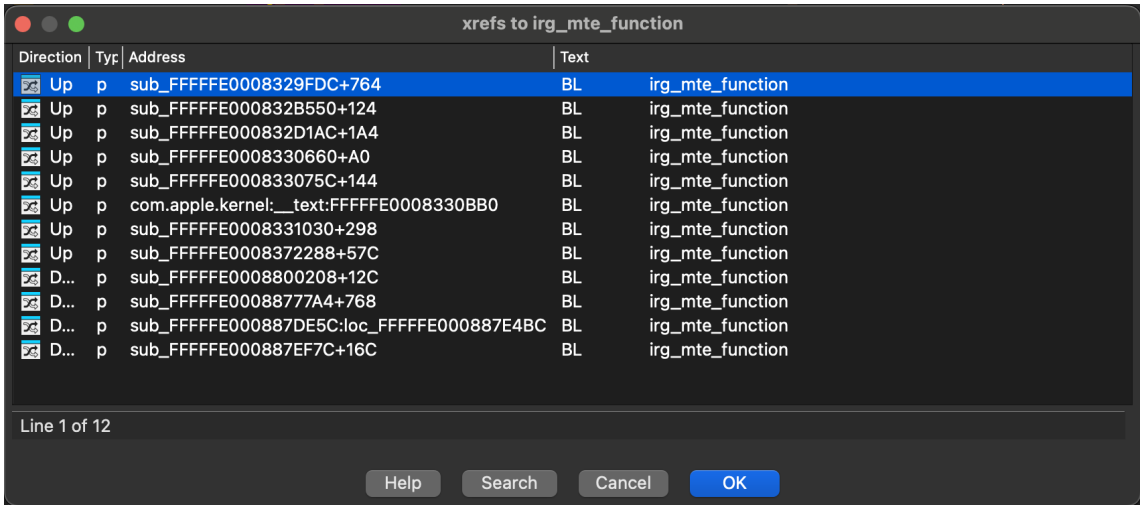


Figure 1: IDA Pro cross-reference view showing the 12 direct call sites (`BL`) to `irg_mte_function`.

Path-sensitive analysis of the direct caller paths found no consistently applied synchronization mechanism spanning tag initialization and the subsequent publication/update operation. Only `sub_FFFFFE0008331030` contained an explicit `DMB ISH`. The remaining analyzed paths relied on atomic RMW operations without an identified end-to-end acquire/release or lock-based publication protocol.

Accordingly, the static analysis does not establish a consistent happens-before relation across the tag-store–publication boundary. Under concurrent publication and consumption, this is consistent with a potential TOCTOU window, unless higher-level synchronization outside the analyzed paths provides the required ordering.

Listing 8 shows the explicit `DMB ISH` in the single barrier-bearing caller context. The fence orders memory accesses on that particular control-flow path before the subsequent shared-state update. Equivalent explicit barrier coverage was not observed on the remaining

analyzed helper-to-publication paths. The relevant result is that explicit barrier-based ordering is path-specific rather than applied consistently across all direct callers of the tagging helper.

```
0xFFFFFE00083310E8: LDR             X10, [X8,#0x10]
0xFFFFFE00083310EC: LDR             X11, [X8,#0x30]
0xFFFFFE00083310F0: DMB             ISH ; ensure memory operations complete
0xFFFFFE00083310F4: LDR             X11, [X11,#8]
0xFFFFFE00083310F8: ADD             X11, X11, #4
0xFFFFFE00083310FC: STUR            X11, [X10,#-8]
```

Listing 8: Caller excerpt from `sub_FFFFFE0008331030` showing the path-specific `DMB ISH` ordering barrier

Table 2 summarizes the path-sensitive analysis of the 11 enclosing functions containing the 12 direct calls to `irg_mte_function`. For each caller, we record whether the analyzed helper-reaching or helper-to-publication path segments contain an explicit architectural barrier or atomic RMW/lock-related operations. The analysis assesses whether the tag-store–publication boundary is spanned by an explicit barrier, a lock-based critical section, or a complete acquire/release publication protocol.

Across the complete direct-caller set (cf. Table 2), three analyzed caller control flows contained `CASA`-class operations, four contained `CASL`, and one resolved caller context contained an explicit `DMB ISH`. Three of the four `CASL` paths also contained `CASA`. Thus, atomic RMW operations are common in the caller graph, whereas explicit barrier coverage is limited to a single analyzed context. Because acquire- or release-qualified RMW instructions provide the relevant ordering only as part of a complete publication/consumption protocol, their presence was not treated as equivalent to synchronization spanning the tag-store–publication boundary. No such complete protocol was identified on the remaining traced direct-caller paths (Table 2; Figure 3).

Table 2: Path-sensitive direct-caller analysis of `irg_mte_function`

Function	Explicit fence	Atomic RMW	Path result
`sub_FFFFFE0008329FDC`	No	Yes	Protocol not identified
`sub_FFFFFE000832B550`	No	Yes	Protocol not identified
`sub_FFFFFE000832D1AC`	No	Yes	Protocol not identified
`sub_FFFFFE0008330660`	No	Yes	Protocol not identified
`sub_FFFFFE000833075C`	No	Yes	Protocol not identified
`sub_FFFFFE0008331030`	`DMB ISH`	Yes	Fence observed; path-specific
`sub_FFFFFE0008372288`	No	Yes	Protocol not identified
`sub_FFFFFE0008800208`	No	Yes	Protocol not identified
`sub_FFFFFE00088777A4`	No	Yes	Protocol not identified
`sub_FFFFFE000887DE5C`	No	Yes	Protocol not identified
`sub_FFFFFE000887EF7C`	No	Yes	Protocol not identified

4.4 Instruction-Level Evidence: Barriers vs. Atomics

Listings 9–11 provide representative instruction-level evidence for the caller-path classification: helper-side tag stores without an explicit architectural barrier, the path-specific `DMB ISH` in `sub_FFFFFE0008331030`, and atomic RMW instructions (`CASA`, `CASL`, `LDADDA`, and `LDADD`) on non-barrier-bearing helper-to-publication path segments.

```
0xFFFFFE000843B638: BTI             c
```

```
0xFFFFFE000843B63C: IRG             X0, X0, X2
0xFFFFFE000843B640: ADD             X8, X0, X1
0xFFFFFE000843B644: EOR             X16, X0, X8
0xFFFFFE000843B648: TST             X16, #0xFFC0000000000000
0xFFFFFE000843B64C: B.EQ            loc_FFFFFE000843B654
0xFFFFFE000843B674: STG             X9, [X9],#0x10
0xFFFFFE000843B6D4: ST2G            X9, [X9],#0x20
0xFFFFFE000843B6D8: ST2G            X9, [X9],#0x20
0xFFFFFE000843B714: RET
```

Listing 9: Excerpt of `irg_mte_function` showing tag generation and tag stores without explicit barriers

```
0xFFFFFE00083310E8: LDR             X10, [X8,#0x10]
0xFFFFFE00083310EC: LDR             X11, [X8,#0x30]
0xFFFFFE00083310F0: DMB             ISH ; explicit ordering barrier
0xFFFFFE00083310F4: LDR             X11, [X11,#8]
0xFFFFFE00083310F8: ADD             X11, X11, #4
0xFFFFFE00083310FC: STUR            X11, [X10,#-8]
```

Listing 10: Caller `sub_FFFFFE0008331030` contains an explicit `DMB ISH` ordering barrier

```
0xFFFFFE000832A174: CASA            X8, X9, [X10]
0xFFFFFE000832A380: CASL            W2, WZR, [X22]
0xFFFFFE000832A79C: LDADDA          W8, W8, [X0]
0xFFFFFE000832AAB0: LDADD           X8, X8, [X9]
```

Listing 11: Acquire- and release-qualified atomic RMW instructions in a caller path without an explicit barrier on the traced helper-to-publication segment

5 Future Work

Two approaches are currently under evaluation. The first approach is to build a custom emulation stack based on QEMU and the open-source *Inferno* project [1]. To make an Inferno-based environment suitable for iPhone 17/A19-class kernels, the most critical adaptation tasks are (i) reconstructing a correct *device tree* and *physical memory/MMIO map* (including reserved regions and interrupt routing), (ii) providing a CPU model that exposes the required architectural features, in particular *MTE* (correct feature bits, tag instructions, and fault behavior) and, where required by the kernel build, *pointer authentication* (PAC), and (iii) implementing a minimal but correct *timer and interrupt-controller* setup to reach stable kernel execution under realistic scheduling conditions. In addition, a functional *early console/logging path* is required to observe boot progress and to instrument the relevant kernel paths. These elements are necessary to emulate the tag-generation and publication behavior with sufficient fidelity to make concurrency conclusions meaningful.

The second approach is to leverage Apple-provided iOS virtualization artifacts that can be extracted from PCC, as described in public write-ups [3], and to use this environment as a more realistic execution substrate for kernel-level testing and instrumentation. Concretely, this would allow targeted stress testing of the identified call paths, measurement of ordering behavior under load, and collection of empirical evidence to confirm or refute the strongly indicated TOCTOU window.

6 Conclusion

This paper presented a static reverse-engineering assessment of the iOS 26.0.1 (Darwin 24.x) kernel MTE tag-generation path on A19-class devices, focusing on the helper routine `irg_mte_function` and its direct caller contexts. The disassembly of `irg_mte_function` and the downstream tagging worker `sub_FFFFFE000843B738` shows no explicit architectural ordering barriers (`DMB/DSB/ISB`) within the helper bodies, indicating that any required inter-core publication ordering must be provided by the surrounding caller context rather than by the helpers themselves.

Across the 12 direct call sites resolved to 11 enclosing caller functions, path-sensitive control-flow analysis identified only one barrier-bearing caller context, `sub_FFFFFE0008331030`, containing an explicit `DMB ISH`. The remaining traced helper-to-publication path segments contained no `DMB`, `DSB`, or `ISB`. Although those paths contained atomic RMW operations, including acquire- and release-qualified variants, no complete acquire/release publication protocol or lock-based critical section spanning both the helper invocation and the relevant shared-state publication was identified. Consequently, the analyzed direct-caller paths do not show a consistently established barrier-, lock-, or identified acquire/release-based happens-before relation across the tag-store–publication boundary. This supports a potential TOCTOU hypothesis under concurrent publication and consumption, subject to any higher-level serialization outside the traced paths.

Finally, we characterized static trigger preconditions for the bulk tagging paths, including a per-context flag gate (byte field at offset `+0x3C`) and a global enable/count parameter (`dword_FFFFFE0007C0E250`) with both read and write references. This study is limited to static analysis, no dynamic validation on MTE-capable A19 hardware or Apple SRD devices was performed, and therefore no exploitability claim is made. Future work should confirm whether the identified call paths are exercised in security-relevant runtime scenarios, and whether the publication/consumption boundary is protected by higher-level synchronization under realistic scheduling and workload conditions.

Acknowledgements

The author would like to thank Andreas Renz for reviewing this paper and providing helpful feedback.

References

[1] ChefKissInc, “Inferno,” GitHub repository, n.d., accessed: 2026-03-02. [Online]. Available: https://github.com/ChefKissInc/Inferno

[2] MITRE. (2026, Jan.) CWE-362: Concurrent execution using shared resource with improper synchronization (’race condition’). The MITRE Corporation. CWE Version 4.19.1. Page last updated: 2026-01-21. [Online]. Available: https://cwe.mitre.org/data/definitions/362.html

[3] wh1te4ever, “super-tart-vphone-writeup,” GitHub repository, n.d., accessed: 2026-03-02. [Online]. Available: https://github.com/wh1te4ever/super-tart-vphone-writeup

Attachment

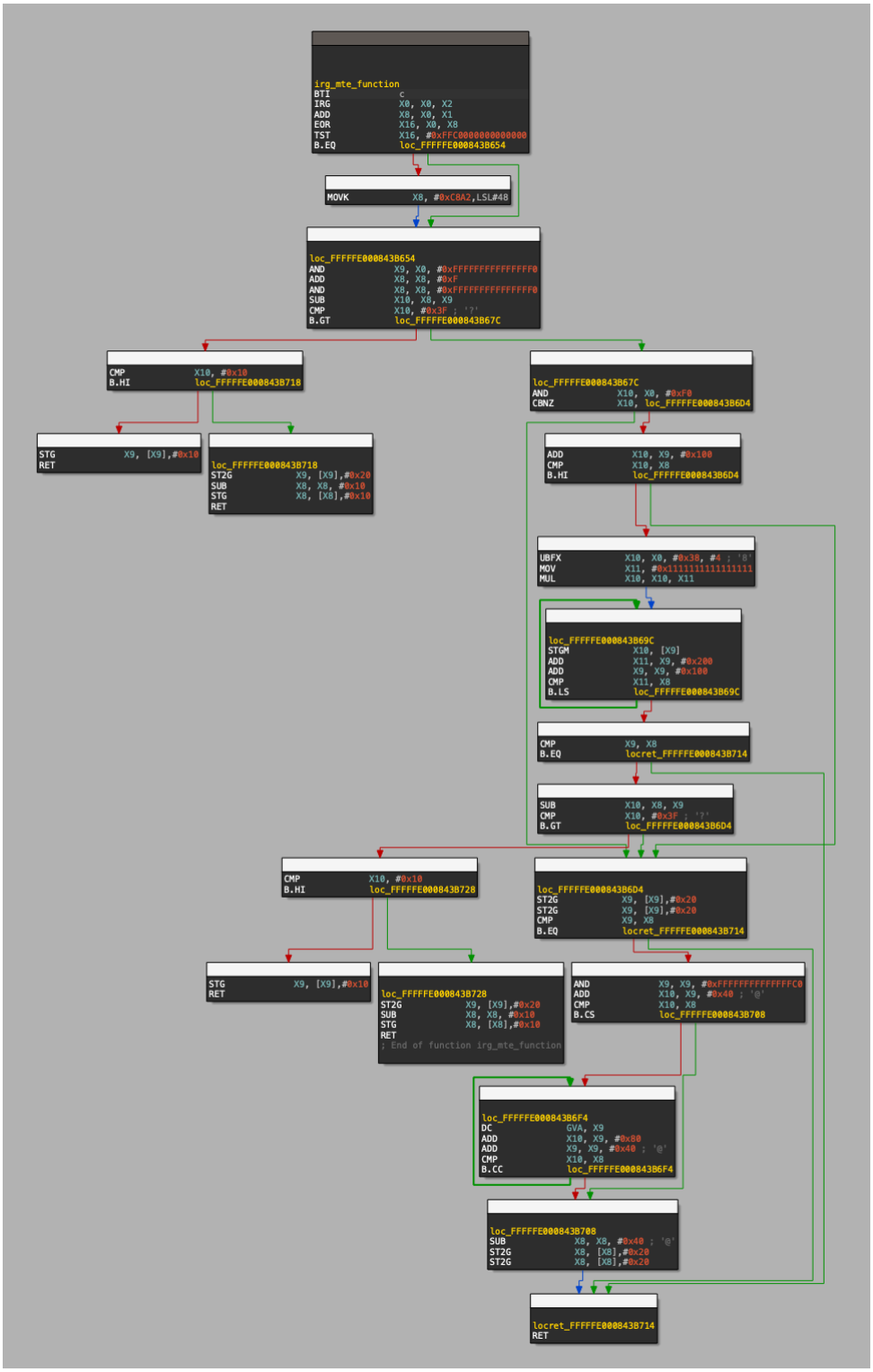


Figure 2: IDA Pro control-flow graph view of `irg_mte_function`, illustrating the internal branching structure and tag-store sequences assessed during static analysis.



Figure 3: Caller-path graph highlighting the direct `BL` call sites to `irg_mte_function` and the helper-reaching and post-call basic blocks inspected for ordering barriers, lock-based critical sections, and atomic RMW publication protocols.
Legend: green = analyzed path containing atomic RMW operations with no explicit barrier on the traced helper-to-publication segment; red = barrier-bearing caller path containing `DMB ISH`.